# MATH1062 coursework

ID: 12345678

May 6, 2026

## 1 Python codes

### 1.1 Code for `find_pivot_largest_subscript`

```
  def find_pivot_largest_subscript(tableau):
    """
    Using the largest subscript rule and the minimum ratio test, find the pivot entry in

    Parameters
    ----------
    tableau : array
    The simplex method tableau in basic form

    Returns
    -------
    r, s : integer
    Row and column indexes of the pivot

    Notes
    -----
    Chooses the valid column with the largest index with positive reduced cost,
    then uses the minimum ratio test to choose the pivot row.
    """
    M, N = tableau.shape
    m = M-1  # Number of constraints
    n = N-1  # Number of variables
    #go right to left unlike blands rule
    s = -1
    #scan collumns
    for col in range(n - 1, -1, -1):
        if tableau[-1, col] > 0:
            s = col
            break

    r = -1
    min_ratio = np.inf

    for row in range(m):
        if tableau[row, s] > 0: # validity
            ratio = tableau[row, -1] / tableau[row, s]#finding smallest ratio within co
            if ratio < min_ratio: # use < so it keeps first row index w said ratio if th
                min_ratio = ratio
                r = row

    return r, s
```

### 1.2 Code for `find_pivot_largest_change`

```
1   def find_pivot_largest_change(tableau):
2       """
3   ----Using the largest change rule and the minimum ratio test, find the pivot entry in th
4
5   ----Parameters
6   ----------
7   ----tableau : array
8   ----The simplex method tableau in basic form
9
10  ----Returns
11  ----------
12  ----r, s : integer
13  ----Row and column indexes of the pivot
14
15  ----Notes
16  ----------
17  ----Set final r and final s to -1 at the start to indicate the best column hasnt been fo
18  ----Chooses the column that gives the greatest improvement in the
19  ----objective function, then uses the minimum ratio test to choose
20  ----the pivot row.
21  ----"""
22      M, N = tableau.shape
23      m = M - 1
24      n = N - 1
25
26      largest_change = -np.inf #starts really small
27      final_r = -1
28      final_s = -1
29
30      for col in range(n): #scan for neg col to improve objective func
31          if tableau[-1, col] > 0:#check if i can use col
32              row_for_col = -1
33              min_ratio = np.inf
34              for row in range(m):
35                  if tableau[row, col] > 0:
36                      ratio = tableau[row, -1] / tableau[row, col]
37
38                      if ratio < min_ratio:
39                          min_ratio = ratio
40                          row_for_col = row
41
42              if row_for_col != -1: #found valif col
43                  change = tableau[-1, col] * min_ratio
44                  if change > largest_change:
45                      largest_change = change
46                      final_r = row_for_col
47                      final_s = col
48
49      r = final_r
50      s = final_s
51
52      return r, s
```

# 2 Complexity analysis

## 2.1 Complexity of find_pivot_largest_subscript

The largest subscript rule works by scanning across the objective row to find the rightmost column that can enter the basic variables so highest index. It may need to check all $n$ columns, so takes $O(n)$. After selecting the column, the minimum ratio test is used to find the pivot row, which requires checking all $m$ rows, it takes $O(m)$. Therefore, the overall complexity of the largest subscript rule is

$$O(n + m).$$

But m = n+5 in this case

$$O(n + m) = O(n + (n + 5)) = O(2n + 5) = O(n).$$

So, in this case the largest subscript rule behaves approximately like a linear-time algorithm.

## 2.2 Complexity of `find_pivot_largest_change`

The largest change rule looks at all $n$ columns to determine which one leads to the greatest improvement so takes $O(n)$. For each column, it performs the minimum ratio test across all $m$ rows, which takes $O(m)$ each time. Therefore, the total complexity is

$$O(nm).$$

But in this case m = n + 5

$$O(nm) = O(n(n + 5)) = O(n^2 + 5n) = O(n^2).$$

So, in this case the largest change rule behaves approximately quadratically.

## 2.3 Comparison of Pivot Rules

The largest subscript rule is less taxing on the computer per iteration since it only scans the columns once and performs one ratio test, giving $O(n + m)$. However, the largest change rule is more taxing because it performs a ratio test for every column, leading to $O(nm)$. Therefore the largest change does more work per iteration. However, it could also be more efficient in totality if it requires less iterations than the largest subscript rule.

# 3 Excel analysis

Table 1: Estimated gradient (slope) and intercept values from the plots

| Pivot Rule | Slope ($p_1$) | Intercept ($p_0$) |
|---|---|---|
| Smallest subscript (Bland) (avg) | 1.4568 | -4.8648 |
| Largest subscript (avg) | 1.4073 | -4.7902 |
| Largest change (avg) | 1.6876 | -5.2765 |
| Smallest subscript (Bland) (worst) | 1.5459 | -4.5562 |
| Largest subscript (worst) | 1.4533 | -4.4612 |
| Largest change (worst) | 1.6682 | -4.9684 |

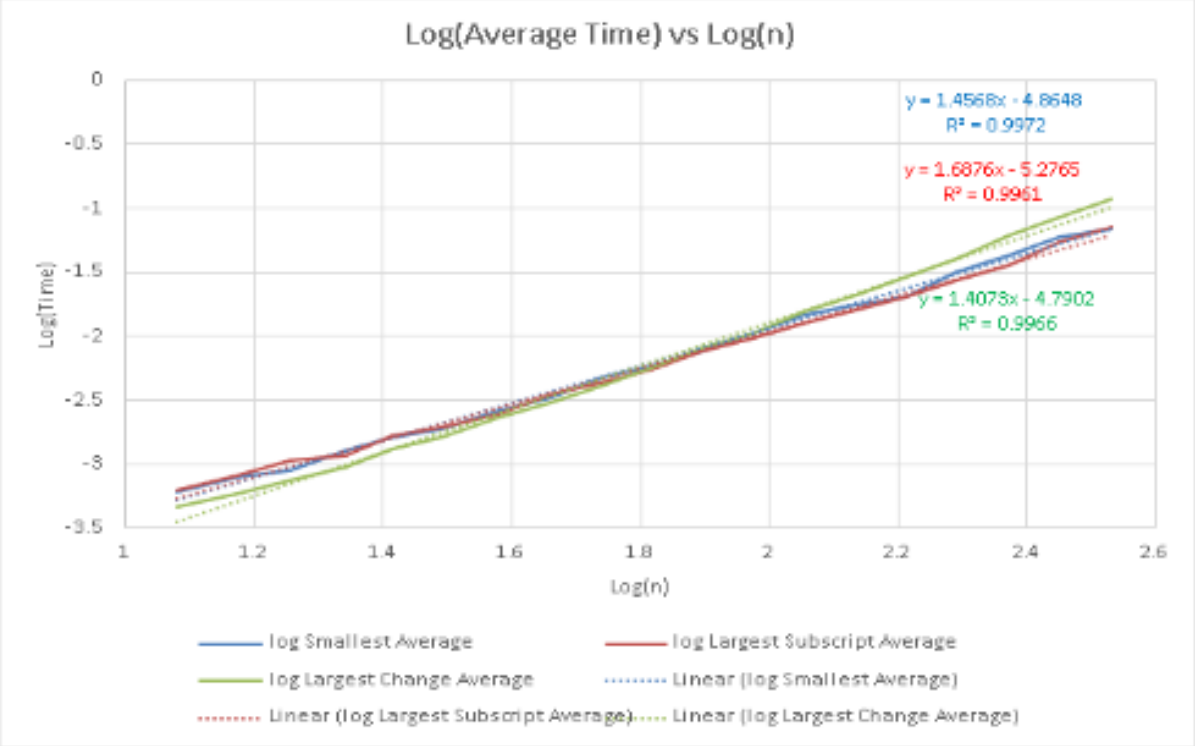


Figure 1: Plot of Log(Average Time) for each pivot rule against log(instance size(n))

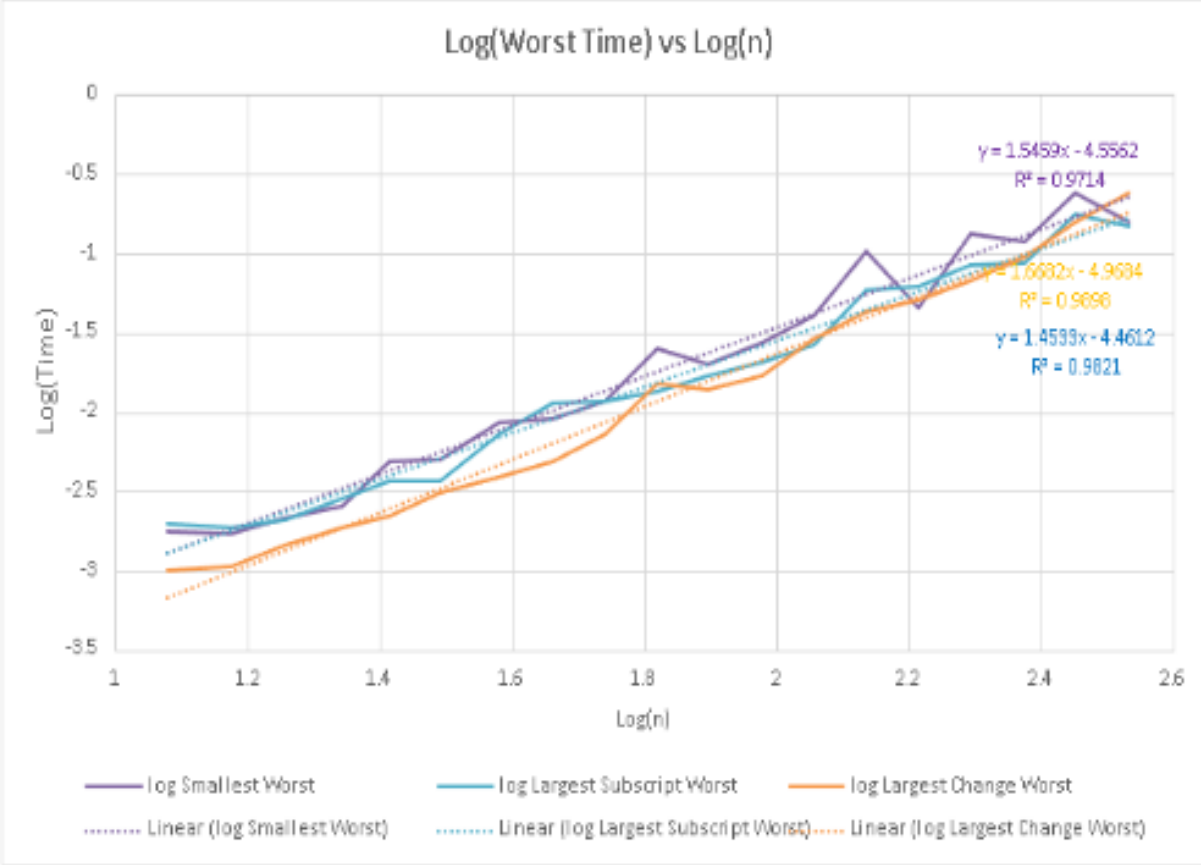


Figure 2: Plot of Log(Worst Time) for each pivot rule against log(instance size(n))

# 4 Efficiency

Referring to Table 1, the largest subscript rule appears to be the most efficient, as it has the lowest slope on average and in the worst case time. This indicates its rate of run-time increases the least out of the three as the instance size increases. On the other hand, the largest change rule had the greatest slope on average and in the worst case which is most apparent in Figure 1, which tells us that its rate of run-time increase is greater than the other two rules, so it is less efficient for larger problems as the time discrepancy becomes more apparent. The difference between the worst case time and the average time for the largest change rule is small, which tells us that its performance is very consistent but always very taxing. Its consistency is very obvious when looking at figure 1 and figure 2 as its lines were the smoothest. The smallest subscript (Bland) rule has a slope in between the other 2 pivot rules, which tells us that it provides consistent performance but is not very efficient. This all matches the complexity analysis, as the largest subscript had complexity $O(n)$ and the largest change rule had complexity $O(n^2)$ which meant the largest change rule had a higher cost per iteration. As previously discussed, although the largest change rule has a higher cost per iteration, this rule may lead to fewer iterations, which could lead to it being more efficient than the other two rules. However, factoring everything in, the largest subscript rule provides the best efficiency while maintaining scalability as problem sizes increase.